

A. Ranking (7 + bonus)

The admission committee for a prestigious high school needs to select the best candidate students, shown in the table, and has decided to rank them according to a combination of their previous marks, weighting English 80% and Math 20%. All marks range from 1 to 10.

<i>English</i>		<i>Math</i>	
Molly:	10.0	Timmy:	10.0
Dolly:	8.0	Dolly:	9.0
Billy:	8.0	Polly:	8.0
Polly:	7.0	Billy:	8.0
Jimmy:	7.0	Jimmy:	6.0
Timmy:	6.0	Molly:	5.0

- 1) Run FA to find the top student according to the given criterion. Specify the accesses that were made.
- 2) Do as above, but with TA.
- 3) **Bonus.** Is there a way to find the same result with fewer accesses, while still guaranteeing correctness?

Solution.

1. FA proceeds by sorted access until it reaches depth 2, finding Dolly in both rankings (4 sorted accesses). Then, it completes the scores for Molly (on *Math*) and Timmy (on *English*) with 2 random accesses. Molly has the top score, computed as $10 \cdot 0.8 + 5 \cdot 0.2 = 9$, while Dolly has 8.2 and Timmy 6.8.
2. TA finds Molly on *English* by sorted access and completes her score with a random access on *Math* (overall 9); similarly, it finds Timmy by s.a. and makes a r.a. on *English* (overall 6.8). The threshold is 10, so another round is needed. It then finds Dolly on *English* by s.a. and completes her score by r.a., and then finds Dolly again by s.a. on *Math*, so no more r.a.'s are needed. The threshold is now 8.2 (same as Dolly's score), and TA can stop. TA makes in total 4 s.a.'s and 3 r.a.'s (note that a slightly different implementation of TA might first read a row entirely, and then make the r.a.'s for the missing pieces, and in this case this would mean saving one r.a. for Dolly, thereby equalling the same cost as FA).
3. A possible strategy is as follows: make one s.a. to *English* (Molly) and one r.a. to *Math* for Molly, thereby finding her score (9). Make one more s.a. to *English* (Dolly); at this point, the threshold is $8 \cdot 0.8 + 10 \cdot 0.2 = 8.4$, which is less than Molly's score, so we can already stop (2 s.a.'s and 1 r.a.). We do not even need to make any s.a. to *Math*, because we know that the upper bound for marks is 10. The guarantees are the same as TA, because we are using a threshold, although we are descending asymmetrically in the rankings.

B. Concurrency (9 points)

Establish whether schedule S , below, belongs to 1) VSR, 2) CSR, 3) 2PL-strict, 4) 2PL, 5) TS-mono.

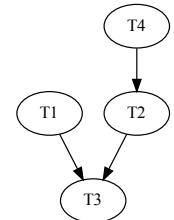
$$S = r_2(X) \quad r_4(W) \quad w_2(X) \quad w_1(Z) \quad r_1(Z) \quad r_3(X) \quad w_1(Y) \quad w_2(W) \quad r_1(Z) \quad w_3(Y).$$

Justify your answers with succinct but clear explanations.

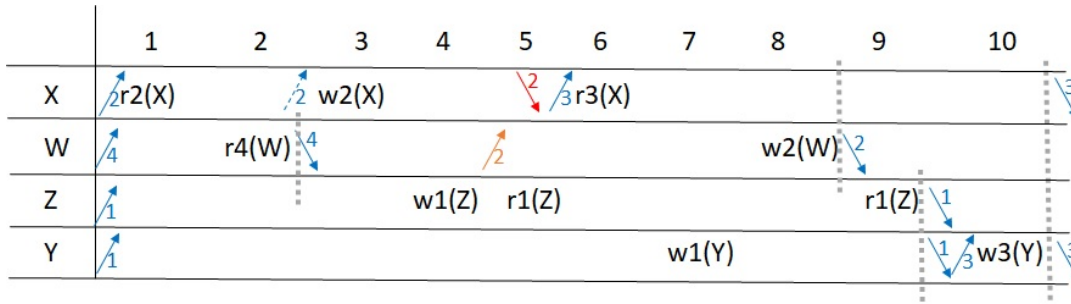
Solution.

Schedule S is in CSR because its resource graph is acyclic. Therefore, S is also in VSR. The schedule is not in 2PL-strict, as shown by the following unsatisfiable set of conditions on resource X :

$$\begin{cases} 8 < XU_2^X & \text{due to strictness, unlock request } XU_2^X \text{ must follow commit time for T2} \\ XU_2^X < SL_3^X & \text{X-lock release must precede S-lock acquisition for } X \text{ as of } w_2^{[3]}(X) \text{ and } r_3^{[6]}(X) \\ SL_3^X < 6 & \text{lock acquisition must precede time of operation } r_3^{[6]}(X) \end{cases}$$



However, it is in 2PL, since Transaction 2 can anticipate the lock request on W before time 6 and then start the releasing phase.



The schedule is not TS-mono, since there are killed transactions:

Operation	RTM(W)	WTM(W)	RTM(X)	WTM(X)	RTM(Y)	WTM(Y)	RTM(Z)	WTM(Z)	Killed transactions
$r_2(X)$	0	0	2	0	0	0	0	0	
$r_4(W)$	4								
$w_2(X)$				2					
$w_1(Z)$								1	
$r_1(Z)$							1		
$r_3(X)$			3						
$w_1(Y)$					1				
$w_2(W)$									2 as $2 < \text{RTM}(W) = 4$
$r_1(Z)$							1		
$w_3(Y)$					3				

C. Triggers (10 points)

The `interval(LeftEnd,RightEnd)` table contains two (non-negative) integer attributes representing the ends of closed time intervals. The `covered(upTo)` table contains a single row with a single integer (initially set to 0) that keeps track of the longest time span from 0 covered by one or more intervals (i.e., all instants from 0 to `upTo` are in some interval in `interval`). For instance, after inserting `[0,10]`, `[10,15]`, and `[14,21]` into `interval`, the value of `upTo` is updated to 21; if the intervals `[25,29]`, `[28,31]`, and `[20,26]` are also added (in any order), then `upTo` should be updated to 31; if `[35,38]` is then added, `upTo` remains unchanged. Users can only insert new time intervals. 1) Design a trigger system that updates `upTo`; 2) discuss termination.

Solution.

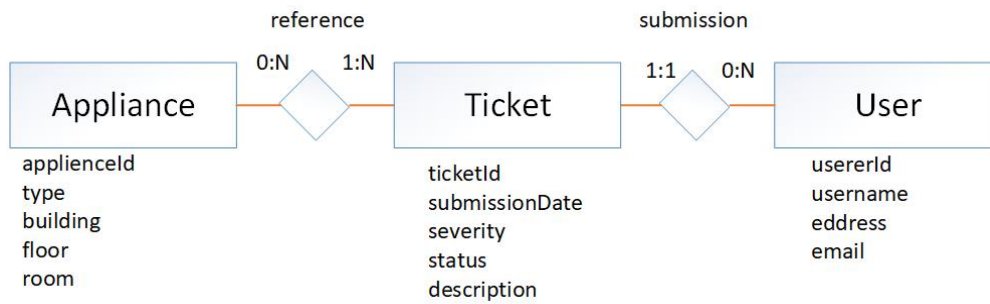
```
CREATE TRIGGER ProcessNewInterval AFTER INSERT ON interval
FOR EACH ROW
  IF (new.LeftEnd <= (SELECT upTo FROM covered)
    AND new.RightEnd > (SELECT upTo FROM covered)) THEN
    UPDATE covered SET upTo = new.RightEnd;
  END IF;
```

```
CREATE TRIGGER UpdateCovered AFTER UPDATE ON covered
FOR EACH ROW
  DECLARE possibleRightEnd INTEGER;
  SELECT coalesce(max(RightEnd),new.upTo) INTO possibleRightEnd
  FROM interval
  WHERE LeftEnd <= new.upTo AND RightEnd > new.upTo;
  IF possibleRightEnd > new.upTo THEN
    UPDATE covered SET upTo = possibleRightEnd;
  END IF;
```

Although the triggering graph is cyclic, since `UpdateCovered` can recursively activate itself, the conditions `RightEnd > new.upTo` and `possibleRightEnd > new.upTo` ensure that a new activation will start only for higher values of `upTo` and that the cycle will then stop at the rightmost interval.

D. JPA (6 points)

An application manages the tickets of a trouble ticketing service. The data are organized according to the following conceptual model.



The application permits the operator to select a user, open his/her list of tickets ordered by submission date descending, select a ticket and check the details of the appliances (one or more) the ticket refers to. Also the operator can select an appliance and check the tickets referring to it, open one such ticket and check its details and the details of the user who submitted it.

Tickets are in the order of millions, appliances are in the order of thousands and users are in the order of hundreds of thousands. Tickets are distributed uniformly across appliances and users.

Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Solution.

See separate file with the solution.